A CASE-STUDY BASED REPORT

ON

CRICKET PLAYER PERFORMANCE PREDICTION USIND RDGE AND LASSO REGRESSION

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in COMPUTER SCIENCE AND ENGINEERING

A.Y.: 2026-27

by

M.V.L. Samhita – Roll No. 2520030509

Sahithi Gotte – Roll No. 2520030499

Under the Esteemed guidance of

Mr. Shaik Asif

Asst.Professor, Dept. of CSE

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

- **Abstract**
- **Introduction**.....

Background.....
Motivation for the Project.....
Overview of the Work Carried Out .....
Scope and Responsible-Use Statement.....

- **Problem Statement**.....

Case-Study Scenario.....
Central Case-Study Question.....
Formal Problem Definition.....
Objectives that Guide the Study .....
Assumptions and Limitations .....

- **Dataset and Exploratory Data Analysis**

About the Dataset .....
Purpose of EDA.....
Preparation Steps Actually Performed.....
EDA Exploration - Implementation.....
Summary of EDA Findings .....

- **Algorithm Selection and Implementation**.....

- **Conclusion**

Signature of the Team members

ABSTRACT

This case study presents a supervised machine-learning system for estimating a batter's runs in an upcoming cricket innings from historical match records. The project frames the task as continuous regression, with `runs_x` as the target variable. The dataset contains 52,492 innings records from 2004 to 2021, representing more than 946 players across 2,854 matches. The modelling workflow compares Ordinary Least Squares (OLS), Ridge Regression, Lasso Regression and Elastic Net.

The process begins with data inspection and cleaning, including removal of a stray index column and a nearly empty field, correction of HTML-encoded dismissal text, handling of missing categorical values, date parsing, and recalculation of strike rate. Exploratory Data Analysis (EDA) examines the distribution of runs, feature scales, outliers and correlations. The target is right-skewed: the reported median is 13 runs and the mean is 22.75 runs. An Interquartile Range (IQR) analysis identifies 2,921 high-score outliers above 74 runs; these are retained because they represent legitimate cricket performances rather than necessarily erroneous records.

Feature engineering constructs 15 numerical historical-form predictors and a categorical match-format feature, using only information from earlier innings. The training script uses an 80/20 train-test split and pipelines that standardise numerical features and one-hot encode match format. On the held-out test set, the reported models achieve MAE values close to 18.13 runs, RMSE values close to 26.03 runs and RZ values close to 0.1385. The results show that regularisation can stabilise linear models, but individual-innings scores remain difficult to predict because of cricket's inherent uncertainty and the skewed target distribution.

The report also explains the deployed Flask-based web application, its player-search and manual-input modes, prediction dashboard, model comparison and uncertainty range. The findings should be interpreted as estimates rather than guarantees. The code review identifies future improvements, particularly chronological evaluation, a simple baseline, format-specific uncertainty bands and additional contextual features.

1. INTRODUCTION

1.1 Background

Machine learning enables a computer to learn statistical relationships from historical examples and use those relationships to estimate outcomes for new observations. Sports analytics is a practical application of this approach because teams and analysts collect structured records describing players, matches and performance. Cricket batting records include information such as runs scored, balls faced, fours, sixes, batting position, match format and dates.

A batter's next innings is influenced by both long-term ability and recent form. Career-level summaries can describe a player's general record, while recent-innings statistics may reflect a temporary change in performance. A model that considers both types of information can offer a more nuanced estimate than relying on a single statistic such as career average.

The Cricket Performance Predictor project uses historical innings data to estimate runs for a future innings. It uses supervised learning because each training observation contains a known target value. It uses regression because the output is a numerical run estimate rather than a discrete category.

1.2 Motivation for the Project

Traditional evaluation can over-rely on one measure. A career average may remain high during a recent slump, while a short run of low scores may not fully represent a player's long-term ability. The project is motivated by the need to combine career history and recent form, account for the format of the match, and present the result in a way that can be explored through a web interface.

The dataset also contains correlated predictors. For example, a player who averages more runs may also average more fours and balls faced. Ordinary linear regression can be sensitive to such multicollinearity. Ridge, Lasso and Elastic Net provide regularised alternatives that penalise coefficient size and may produce more stable models.

1.3 Overview of the Work Carried Out

- Loaded historical cricket data from `bat.csv` and inspected the structure of the available fields.
- Cleaned the dataset by removing irrelevant columns, handling missing categorical values, decoding HTML entities and standardising date and strike-rate fields.
- Explored run distributions, feature distributions, high-score outliers and correlations between historical predictors.
- Created leakage-aware career and recent-form features using previous innings only.
- Built scikit-learn pipelines with numerical standardisation and one-hot encoding of match format.
- Trained and evaluated OLS, Ridge, Lasso and Elastic Net on an 80/20 split.

- Built a Flask backend and browser-based dashboard that supports player lookup and manual recent-score entry.

1.4 Scope and Responsible-Use Statement

The project focuses on estimating runs for an upcoming innings using the available historical data. It is intended as a decision-support and educational analytics tool. It does not determine a player's overall worth, guarantee a score, or replace domain expertise. Match conditions, opposition quality, venue, pitch, weather, team strategy, injury, and the circumstances of an innings can affect the actual outcome but are not fully represented by the current feature set.

The output should therefore be read as a statistical forecast with uncertainty. It should not be used as the sole basis for selection, contracts, or other consequential decisions. Users should consider the model's error metrics and limitations, particularly when comparing players from different formats or with limited historical records.

2. PROBLEM STATEMENT

2.1 Case-Study Scenario

A cricket selection panel is discussing whether Player A should be selected for an upcoming ODI. One selector highlights a career batting average of 38.40, suggesting consistent long-term ability. Another focuses on a recent five-innings average of 14.20, suggesting a decline in form. Both statistics are useful, but neither alone captures the full context.

The project applies regression models to historical career and recent-form features to produce a run estimate. The resulting forecast can serve as one additional perspective during analysis. It cannot settle a selection decision by itself, because a single innings has substantial random variation and the current system does not model every match condition.

2.2 Central Case-Study Question

Can historical career statistics and recent-form indicators be used to estimate a batter's runs in a future innings, and how do OLS, Ridge, Lasso and Elastic Net compare when evaluated on unseen cricket innings?

2.3 Formal Problem Definition

Let X represent the input feature vector constructed from a player's previous innings and the selected match format. Let y represent the observed target `runs_x`, the runs scored in the current innings. The objective is to learn a function f(X) that predicts $\hat{y}$, an estimate of the runs for an innings not used to fit the model.

The training data contains labelled examples, so this is supervised learning. The target is numeric, so this is a regression problem. The central modelling constraint is temporal: the features for an innings must be based only on information available before that innings. Using statistics from the innings being predicted would leak the answer into the inputs and make evaluation misleading.

2.4 Objectives

1. Inspect the cricket dataset and understand its size, fields and quality.
2. Clean text, dates and missing categorical values and verify basic consistency checks.
3. Use EDA to understand run distribution, feature ranges, unusual values and correlations.
4. Engineer career and recent-form predictors using prior innings only.
5. Train OLS, Ridge, Lasso and Elastic Net regression models using a reproducible split.
6. Compare models using MAE, MSE, RMSE and RZ.
7. Present forecasts and supporting information through a web application.
8. Explain limitations and identify practical improvements to make future evaluation more trustworthy.

2.5 Assumptions and Limitations

- The historical dataset is assumed to be sufficiently accurate for modelling after the documented cleaning steps.
- Past performance contains useful information about future performance, but the relationship is imperfect.
- High scores above 74 are treated as valid observations and retained, rather than automatically removed as data errors.
- Linear models may underfit the complex and skewed distribution of cricket scores.
- The current random split may place different innings from the same player in both training and test sets; this can make a future-prediction evaluation optimistic.
- The available data does not fully represent opposition, venue, pitch, weather, innings context or player fitness.
- The reported evaluation values describe the supplied project report and training configuration; they are not a guarantee of future performance.

3. DATASET AND EXPLORATORY DATA ANALYSIS

3.1 About the Dataset

The project uses the cricket batting dataset file `bat.csv`, described as one row per batter per innings. The broader dataset directory also contains `ball.csv` and `match.csv`, although the predictor training pipeline described in the code review reads `bat.csv`. The dataset period is 2004–2021.

Dataset property	Reported value
Innings records loaded	52,492
Players represented	1,571 players audited; 946+ players referenced in project summary
Matches represented	2,854
Original columns	28
Target variable	`runs_x` — runs scored in the innings
Target median	13 runs
Target mean	22.75 runs
Outliers identified using IQR	2,921 observations above 74 runs (5.56%)

The source materials refer to a 28-column original dataset, described as 19 numerical and 7 categorical features in one audit summary. The final modelling frame is a derived table containing 15 numerical historical features plus `matchtype`, with `runs_x` as the target. These are different stages of the pipeline and should not be confused: raw dataset columns are not identical to model input features.

3.2 Data Inspection

The inspection stage establishes the size and types of the raw data, checks whether fields contain missing values, and identifies columns that are not useful for modelling. The original data includes identifiers, player and match names, dates, match format, batting statistics, dismissal information and a nearly empty fall-of-wicket field. Some columns are useful for grouping and sorting, even when they are not used directly as predictors.

The player and match identifiers help group each batter's innings and establish chronological order. The target `runs_x` records the current innings score. Historical feature engineering then transforms the innings table into a model-ready training frame.

3.3 Data Cleaning and Preprocessing

Issue found	Action documented	Reason
Stray `Unnamed: 0` column	Dropped the stray index column	It is a saved DataFrame index, not a meaningful cricket statistic.
`fall_of_wicket` was 99.92% missing	Dropped the column	A nearly empty field provides very little usable information.

Issue found	Action documented	Reason
HTML entities in `how_out`	Decoded HTML entities into readable text	Makes dismissal descriptions readable and consistent.
Missing `matchtype` values (37 rows)	Filled with `unknown`	Preserves rows while making missing categories explicit.
Missing `playing_role` values (1,543 rows)	Filled with `unspecified`	Avoids leaving the categorical field blank.
Missing `how_out` values (9 rows)	Filled with `unspecified`	Creates a consistent category for missing dismissal information.
`match_number` stored as float with 635 missing values	Imputed missing values with 0 and cast to integer	Provides a consistent integer representation.
`start_date` stored as text	Parsed to datetime	Supports chronological sorting and calculating gaps between innings.
Strike-rate field precision / zero balls	Recalculated as runs divided by balls times 100; set to 0 when balls = 0	Ensures a consistent formula and avoids division by zero.
Boundary and duplicate checks	Verified boundary arithmetic and reported zero duplicate rows	Provides basic integrity checks across all 52,492 rows.

The data-cleaning audit reports that 9,411 `how_out` entries contained HTML entities and that 828 valid innings had zero balls faced. These details are important because a clean dataset must not introduce invalid values when calculating derived statistics.

3.4 Feature Engineering

The most important modelling design choice is that the model should predict an innings using only information from earlier innings. For each player, innings are sorted by `start_date`, `match_id` and `id`. The historical statistics are shifted by one row before expanding or rolling calculations are applied. This means that the current innings score is not included in the features used to predict that same innings.

The training-frame builder removes the first innings for each player because no previous history exists for it. It fills a missing `days_gap` with zero and drops rows with missing features. The resulting modelling frame contains 15 numerical historical features and the categorical `matchtype`, with `runs_x` as the target.

Feature	Meaning	How it is derived
`n_innings`	Prior experience	Count of innings played before the current innings.
`car_runs`	Career runs average	Expanding mean of previous `runs_x` values.
`car_SR`	Career strike rate	Expanding mean of previous strike rates.
`car_balls`	Average balls faced	Expanding mean of previous balls faced.
`car_order`	Usual batting position	Expanding mean of previous batting orders.
`car_fours`, `car_sixes`	Career boundary rate	Expanding means of previous fours and sixes.
`car_max`	Highest score so far	Expanding maximum of previous innings runs.

Feature	Meaning	How it is derived
`last3_runs`	Recent scoring form	Mean of up to the previous three innings.
`last5_runs`	Recent scoring form	Mean of up to the previous five innings.
`last5_SR`, `last5_balls`	Recent strike rate and balls	Rolling means over the previous five innings.
`last5_fours`, `last5_sixes`	Recent boundary form	Rolling means over the previous five innings.
`days_gap`	Time since previous innings	Difference in days between consecutive innings.
`matchtype`	Match format	Categorical field, later one-hot encoded.

For predictions on an existing player, the system computes the same feature definitions from that player's available history. The code uses a fixed 14-day gap for the next innings because the date of the next fixture is unknown. In manual mode, some features are estimated using helper linear models because a user may provide only a list of recent scores.

3.5 Purpose and Implementation of EDA

Exploratory Data Analysis is performed before modelling to understand what the dataset looks like and to identify issues that could affect learning. The project uses a target distribution plot, distributions of historical features, a correlation heatmap, a model-performance comparison plot and a standardised-coefficient plot.

3.5.1 Distribution of Innings Runs

The target distribution is strongly right-skewed. The reported median is 13 runs, while the mean is 22.75 runs. The mean being higher than the median is consistent with a relatively small number of high scores pulling the average upward. The project reports that 10.96% of innings were ducks (zero runs) and 2.62% reached 100 runs or more.

In the target distribution graph, the horizontal axis groups innings into run bands such as 0–9, 10–19, 20–29, 30–39, 40–49, 50–74, 75–99 and 100+. The vertical axis represents the number of innings in each group. The graph shows that low scores occur much more frequently than centuries.

3.5.2 Historical Feature Distributions

The historical features have different numerical scales. For example, the number of prior innings can extend above 700, while batting order usually falls between 1 and 11. Features such as career runs, balls faced, strike rate and recent-form averages therefore cannot be compared fairly by raw magnitude alone. This motivates standardising numeric features inside the modelling pipeline before applying regularisation.

3.5.3 Correlation Matrix and Multicollinearity

The correlation heatmap compares the 16 historical form features and the target. The project reports the following high positive correlations:

- `car_runs` and `car_fours`: $r = +0.93$
- `last5_runs` and `last5_fours`: $r = +0.91$

- `car_runs` and `car_balls`: r = +0.85

These relationships are understandable: innings with more runs often include more boundaries and more balls faced. However, highly correlated predictors can make individual coefficients unstable in an ordinary linear model. Ridge and Elastic Net are therefore reasonable candidates because their penalties can reduce coefficient magnitudes. Correlation does not by itself establish causation.

3.5.4 Outlier Analysis

The project used the Interquartile Range (IQR) method to identify 2,921 target observations above 74 runs, corresponding to 5.56% of the innings dataset. Since the target median is only 13 runs, these scores form the upper tail of the distribution. They include high-quality innings and centuries, which are valid cricket outcomes rather than necessarily data-entry errors.

The project retained these observations so the model would not be trained only on ordinary or low scores. Removing every high score would change the task and could reduce the model's exposure to elite performances. The report distinguishes an observation that is statistically unusual from an observation that is erroneous.

3.6 EDA Findings Summary

Finding	Interpretation for modelling
Median 13; mean 22.75	Target is right-skewed; large scores pull the mean upward.
10.96% ducks; 2.62% centuries	The target has many low outcomes and a small high-score tail.
2,921 IQR outliers above 74 runs	High scores are retained as plausible sporting outcomes.
Strong correlations among career and recent-form variables	OLS coefficients may be unstable; regularisation is worth testing.
Features have very different ranges	StandardScaler is used for numerical predictors.
Match format varies (ODI, T20, Test)	Matchtype is encoded categorically so the model can account for format differences.

4. ALGORITHM SELECTION AND IMPLEMENTATION

4.1 Learning Task and Feature Inputs

The target `runs_x` is a numerical value, so the project is a regression task rather than classification. The input variables are historical features describing experience, career-level batting performance, recent form and match format. The models are supervised because the training set includes the known runs scored in each historical innings.

4.2 Train/Test Split and Preprocessing Pipeline

The training script uses `train_test_split` with `test_size=0.20` and `random_state=42`. The reported split contains 40,736 training innings and 10,185 test innings. The test set is held out from fitting and used to compare predictions with actual runs.

Each model is wrapped in a scikit-learn Pipeline. Numeric features are processed with `StandardScaler`, which transforms them to a common scale using the training data. The categorical `matchtype` field is processed with `OneHotEncoder(handle_unknown="ignore")`, which represents each format through indicator columns and safely handles an unseen category. The preprocessing is part of the pipeline, so the same transformations are applied during training and prediction.

One limitation is that the current split is random rather than chronological. Different innings from the same player and different time periods may occur in both sets. For a strict future-forecasting claim, a time-based split would provide a more realistic test of how the model performs on later matches.

4.3 Ordinary Least Squares (OLS)

Ordinary Least Squares is the standard linear-regression baseline. It chooses coefficients that minimise the sum of squared differences between actual and predicted training targets. OLS is fast and easy to interpret, but it does not penalise large coefficients. When predictors are highly correlated, small changes in data can produce unstable coefficient estimates.

4.4 Ridge Regression

Ridge Regression adds an L2 penalty to the least-squares objective. The penalty discourages large coefficient values, which can improve stability when predictors contain overlapping information. Ridge generally shrinks coefficients toward zero but does not usually force them to become exactly zero.

The project uses `alpha = 1.0` for Ridge. It is used as the headline model in the web application, and the project saves its coefficients and residual-based uncertainty band. The model's headline role is a design choice; it should not be interpreted as proof that Ridge is meaningfully superior to every alternative.

4.5 Lasso Regression

Lasso Regression adds an L1 penalty based on the absolute values of coefficients. In addition to shrinking coefficients, Lasso can set some coefficients exactly to zero. This can simplify a model by reducing the influence of less useful predictors. However, when features are strongly correlated, Lasso may select one feature and suppress another similar feature.

The project uses `alpha = 0.01` for Lasso. The regularisation strength controls the balance between fitting the training data and shrinking the coefficients.

4.6 Elastic Net

Elastic Net combines L1 and L2 regularisation. It offers a compromise between Ridge's coefficient shrinkage and Lasso's ability to produce sparse coefficients. The project uses `alpha = 0.01` and `l1_ratio = 0.5`, which combines both penalties in equal proportions according to the configured ratio.

4.7 Evaluation Metrics

Metric	Definition	How to interpret it
MAE	Mean Absolute Error: average absolute difference between actual and predicted runs.	Lower is better; easy to interpret as average error in runs.
MSE	Mean Squared Error: average squared prediction error.	Lower is better; large errors receive stronger penalties because errors are squared.
RMSE	Square root of MSE.	Lower is better; reported in runs, like the target.
RZ	Coefficient of determination, measuring variance explained relative to a mean-prediction baseline.	Higher is generally better; 1 is perfect, 0 is comparable to predicting the mean, and a negative score is worse than that baseline.

4.8 Results and Comparison

The supplied project report gives the following test-set metrics. These values are reproduced from the report rather than recalculated from the code, because the uploaded code archive does not include the original `bat.csv` data file or the trained model artefacts.

Model	MAE (runs)	MSE	RMSE (runs)	R^2
OLS	18.1389	677.7331	26.0333	0.1385
Ridge (α = 1.0)	18.1387	677.7294	26.0332	0.1385
Lasso (α = 0.01)	18.1336	677.7582	26.0338	0.1384
Elastic Net (α = 0.01, l1_ratio = 0.5)	18.1304	677.7955	26.0345	0.1384

Elastic Net has the lowest reported MAE at 18.1304 runs, while Ridge has the lowest reported MSE and RMSE by a very small margin. The reported RZ values are nearly identical at approximately 0.1384–0.1385. These differences are extremely small, so the results do not support a strong claim that one model decisively outperforms the others.

An RZ of approximately 0.1385 means the model explains around 13.85% of the variation in the held-out target relative to the test-set mean baseline. This is limited explanatory power, but it is not unexpected for predicting a single innings score, which can be affected by random events such as dismissal, bowling quality, match conditions and the player's role. The MAE indicates that the typical absolute miss is about 18 runs.

4.9 Real-World Case Study: Team Selection for Player A

The project report provides an illustrative selection scenario. Player A has a career average of 38.40 runs but has averaged only 14.20 runs over the last five innings. The following forecasts are the example values documented in the supplied report:

Model	Predicted runs	Interpretation in the report
OLS	28.50	Potentially unstable due to multicollinearity between career runs and fours.
Ridge (α = 1.0)	22.40	Balances the career baseline with the recent slump.
Lasso (α = 0.01)	21.80	Places stronger emphasis on recent-form signals.
Elastic Net (α = 0.01)	22.10	Compromise between Ridge and Lasso.

This scenario illustrates how different models can combine the same evidence in different ways. The forecasts are not observed future scores and should not be treated as an objective selection verdict. A selection panel should also consider the player's role, match conditions, opposition, fitness and other evidence unavailable to the model.

4.10 Web Application Design

The project turns the trained models into an interactive web application using Flask for the backend and a single-page HTML/CSS/JavaScript frontend. The backend loads saved model pipelines and supporting data at startup; it does not retrain the models for each request. Training and serving are therefore separate phases.

4.10.1 Project Structure

File or folder	Purpose
`README.md`	Setup instructions and quick-start guide.
`requirements.txt`	Python dependencies, including Flask, pandas, NumPy, scikit-learn and joblib.
`features.py`	Shared data cleaning and feature-engineering functions.
`train_models.py`	Loads the dataset, creates the modelling frame, trains and evaluates four models, and saves artefacts.
`app.py`	Flask server, API routes, prediction logic and explanation calculations.
`static/index.html`	Frontend layout, user inputs, chart and results dashboard.
`data/bat.csv`	Input dataset placed in the data folder by the user.
`model/`	Generated model pipelines, player and innings data, and metadata.

4.10.2 Backend API

Route	Method	Purpose
`/`	GET	Serves the web interface.
`/api/meta`	GET	Returns model metrics, uncertainty information, coefficients, counts and formats.
`/api/players?q=...`	GET	Searches for players by name.
`/api/player/<id>`	GET	Returns a player summary and recent innings.
`/api/predict`	POST	Returns predictions and supporting explanation data.

The prediction endpoint supports two modes. In player mode, the app uses the selected player's actual historical innings to build the next-innings features. In manual mode, the user enters a list of recent scores and may optionally provide career average, innings played, batting position and strike rate. Helper regression models estimate some features that cannot be calculated from the short score list alone, making manual mode less reliable than player mode.

4.10.3 Frontend and User Experience

The frontend provides two tabs: "Pick a player" and "Enter scores". Users select ODI, T20 or Test format. Player search calls the backend after a short debounce, reducing repeated requests while the user types. The interface validates inputs, disables the predict button during an active request, and displays clear error messages.

The results panel displays the Ridge forecast as the headline number, a plausible range based on historical Ridge residual percentiles, a recent-innings chart, the predictions from all four models, a driver breakdown and a table of recent innings for player mode. The dashboard also displays model error metrics so users can judge the forecast rather than seeing only one confident-looking number.

4.10.4 End-to-End Request Flow

1. The user searches for a player or enters recent scores and selects a match format.
2. The browser sends a player-search request or prepares the manual prediction input.
3. The Flask backend validates the request and builds the 15 historical features.
4. The four saved pipelines apply numeric scaling and match-format encoding, then generate predictions.
5. The backend clamps negative run forecasts to zero, calculates the Ridge-based driver contributions and constructs the uncertainty range.
6. The frontend renders the headline prediction, chart, model comparison, drivers and recent innings information.

5. RESULTS, DISCUSSION AND RESPONSIBLE INTERPRETATION

5.1 Main Results

The reported model metrics are close to one another. Elastic Net produces the smallest MAE, while Ridge has the smallest MSE and RMSE. The RZ values are almost identical. Therefore, the practical differences among the four models are minor under the reported configuration.

The metrics also show that predicting a single innings remains challenging. An MAE near 18 runs means a forecast can differ substantially from the actual score. The model should be used to describe a plausible statistical estimate, not to promise a particular result.

5.2 Why Regularisation Is Relevant

The EDA reports high correlations between career runs and career fours, recent runs and recent fours, and career runs and balls faced. Such relationships mean that several predictors may convey overlapping information. OLS can assign unstable weights among correlated features. Ridge reduces coefficient magnitudes; Lasso can remove some predictors by setting their coefficients to zero; Elastic Net combines the two approaches.

5.3 Interpreting Coefficients

The coefficient analysis in the project identifies `car_runs` as a major positive driver, with standardised coefficient values reported in the approximate range +1.17 to +1.35. `car_order` is reported as a major negative driver with a coefficient around −1.00. The sign of a coefficient describes the direction of association after accounting for the other model inputs. It does not prove that changing a feature causes a player's score to change.

Because the numerical predictors are standardised, coefficient sizes are interpreted on the transformed scale. In addition, correlated features can share or redistribute the model's explanatory signal. Coefficient plots should therefore be used as a description of the fitted model, not as an independent causal analysis.

5.4 Uncertainty Range

The system calculates an uncertainty band from the 10th and 90th percentiles of Ridge residuals on the test set. This range is intended to cover the middle 80% of historical errors under the residual distribution used during training. The lower end is floored at zero because negative run totals are impossible.

The current implementation uses one global residual band for all players and formats. The project review notes that this is a limitation because Test, ODI and T20 scores can have different distributions and variability. The range should not be interpreted as a calibrated guarantee for every individual player.

5.5 Strengths of the Project

- Leakage-aware feature engineering shifts historical statistics so that the current innings is not used to predict itself.
- Training and serving share the same feature definitions, reducing the risk of inconsistent preprocessing between model development and the live app.
- The comparison of OLS, Ridge, Lasso and Elastic Net gives a defensible baseline and regularisation study.
- The interface exposes metrics and uncertainty rather than showing only a single forecast.
- Player mode uses real historical data, while manual mode supports hypothetical inputs.
- The application includes request validation, clear error handling, escaped text in the frontend, accessible interface attributes and reduced-motion support.

6. LIMITATIONS AND FUTURE ENHANCEMENTS

6.1 Limitations

- Random splitting can place earlier and later innings from the same player in both training and testing, potentially making evaluation optimistic for future prediction.
- The target is strongly right-skewed; linear models may struggle with many low scores and a small number of very high scores.
- Fixed hyperparameters are used without cross-validation, so the selected regularisation strengths may not be optimal.
- The current feature set does not fully include opposition, venue, home/away status, innings number, pitch, team strength or player fitness.
- A single global uncertainty band is applied across formats with different scoring distributions.
- Manual mode estimates several inputs through helper models trained across mixed formats, so it is less reliable than player mode.
- The current model comparison does not include a simple career-average baseline, making it harder to quantify the added value of machine learning.
- The application is intended for local use in its current form; debug mode must not be enabled in a public deployment.

6.2 Recommended Improvements

1. Use a chronological train/test split: train on earlier matches and test on later matches to better simulate real future forecasting.
2. Add a naive baseline that predicts a player's career average or another simple historical estimate, then compare it with the machine-learning models.
3. Use cross-validation or GridSearchCV/RidgeCV/LassoCV/ElasticNetCV to tune regularisation strengths systematically.
4. Evaluate a tree-based model such as HistGradientBoostingRegressor and consider log- or square-root transformations of the target.
5. Estimate uncertainty bands separately by match format or use a more formal method such as quantile regression or conformal prediction.
6. Add contextual features such as opposition, venue, home/away status and innings context if the dataset supports them.
7. Improve manual-mode validation and clearly communicate that manually estimated features make its forecasts less reliable.
8. Add automated tests for feature leakage, API validation and missing input handling; pin dependency versions for reproducible deployment.

9. Disable debug mode and use a production WSGI server if the app is deployed beyond localhost.

7. CONCLUSION

This case study developed a supervised regression workflow for predicting cricket batting runs from historical innings records. It covered data inspection, cleaning, exploratory analysis, temporal feature engineering, preprocessing, model fitting, evaluation and deployment through a Flask-based web application.

The dataset contains 52,492 innings records from 2004–2021. EDA found a strongly right-skewed target with a median of 13 runs and a mean of 22.75 runs. The IQR method identified 2,921 high-score observations above 74 runs, which were retained because high cricket scores are plausible performances rather than automatic data errors. The correlation analysis found substantial relationships among career and recent-form features, supporting the investigation of regularised regression.

OLS, Ridge, Lasso and Elastic Net achieved similar reported results on the held-out test set, with MAE values around 18.13 runs, RMSE around 26.03 runs and RZ around 0.1385. Elastic Net had the lowest MAE, while Ridge had the lowest MSE and RMSE by a very small margin. These results suggest that no model decisively dominates under the current settings. They also reinforce that individual innings are inherently difficult to forecast.

The web application makes the model accessible through player selection or manual recent-score input and presents model comparisons, explanatory drivers and an uncertainty range. However, the results should be treated as estimates, not guarantees. A chronological evaluation split, baseline comparison, hyperparameter tuning, format-specific uncertainty estimates and richer match-context features are the most important next steps for improving reliability.

REFERENCES

1. Cricket Performance Predictor — Detailed Project Review and Walkthrough. Supplied project review document describing the codebase, cleaning, feature engineering, models, Flask API, frontend, limitations and run instructions.

2. Cricket Performance Player Prediction: Predicting Cricket Player Performance from Previous Match Records using Ridge, Lasso, and ElasticNet Regression. Supplied project report containing dataset audit, EDA findings, model metrics, coefficient discussion and case-study example.

3. Scikit-learn documentation: LinearRegression, Ridge, Lasso, ElasticNet, Pipeline, ColumnTransformer, StandardScaler, OneHotEncoder, train_test_split and regression metrics. Add the exact URLs and access date if required by the course.

4. Dataset source: `ML_DATASET` / `bat.csv` as referenced in the supplied project documentation. Add the original dataset provider or repository URL before final submission if required.

APPENDIX A: RUNNING THE PROJECT

The supplied project review gives the following local run sequence. Execute these commands from the `cricket_app` project directory after placing `bat.csv` inside the `data/` folder:

```
pip install -r requirements.txt
mkdir data  # create this folder if it does not exist; place bat.csv inside
it
python train_models.py
python app.py
```

After starting the Flask server, open `http://127.0.0.1:5000` in a browser. Training must be run before starting the app because the app loads the generated model artefacts at startup. The uploaded code archive does not contain `bat.csv` or the trained model folder, so these must be supplied or generated locally.

APPENDIX B: KEY TERMS

Term	Meaning
Data leakage	Using information from the target or future when constructing inputs, leading to an unrealistically optimistic evaluation.
Lag feature	A feature based on earlier observations, such as the average of a player's previous five innings.
Expanding mean	An average calculated over all available earlier observations.
Rolling mean	An average calculated over a fixed recent window, such as the previous three or five innings.
Multicollinearity	A situation in which input features are strongly related to one another.
Standardisation	Transforming numerical features to a common scale, usually mean zero and standard deviation one.
One-hot encoding	Representing a category such as ODI, T20 or Test with indicator columns.
Residual	The difference between the actual target and the model's prediction.
Regularisation	A penalty added to the model objective to discourage overly large coefficients.
Pipeline	A sequence of preprocessing and modelling steps packaged together for consistent fitting and prediction.